THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

ST445 – Week 2
Managing and Visualising Data

Collecting data from the web

Dr Leonardo Alves Dias



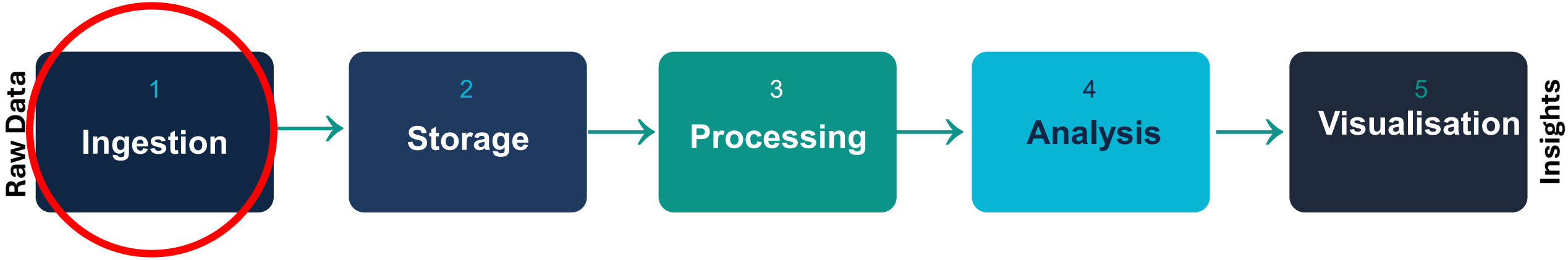
Agenda

- Introduction to Data Ingestion
- Introduction to APIs
- Introduction to Web Scraping

Learning Outcomes

3. Acquire data lawfully and ethically from files, REST APIs or web pages and integrate data from multiple sources through join operations on common keys.

Data Pipeline Components



- A Data Pipeline is a set of processes organised to automate the flow of data from its collection to visualisation. The pipeline consists of sequential stages: **Data Ingestion, Data Storage, Data Processing, Data Analysis, and Data Visualisation.**

- The design of each component impacts the efficiency, scalability, and reliability of the entire pipeline.

DATA INGESTION

What is Data Ingestion?

- Data ingestion is the process of obtaining, importing, and processing data from various sources for immediate use or storage in a database. It's the critical first step in any data pipeline.

- Key Considerations in the context of Big Data:

Volume

How much data needs to be ingested? MB, GB, TB, or PB per day?

Velocity

How fast is data arriving? Real-time streams or periodic batches?

Variety

What types of data sources? Structured databases, logs, IoT sensors, APIs?

Methods of Data Ingestion/Sourcing

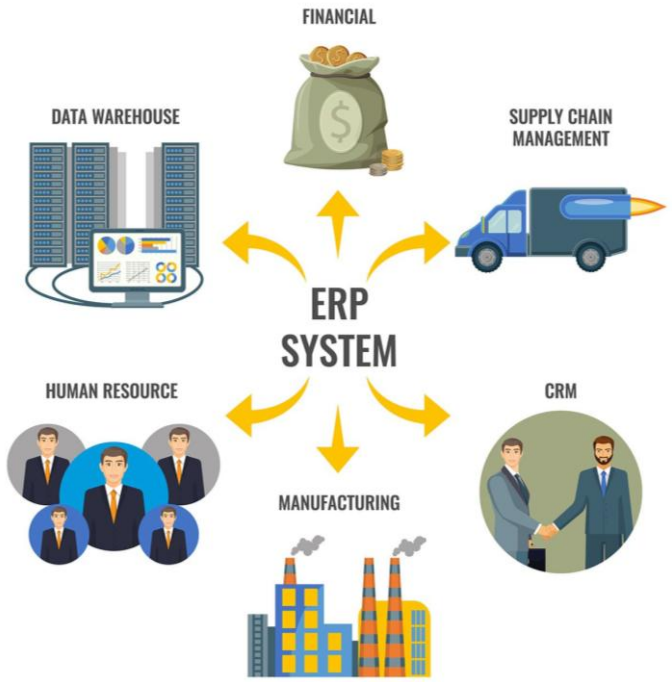


Fig. 1. Internal Business Systems.



Fig. 2. Social Media.

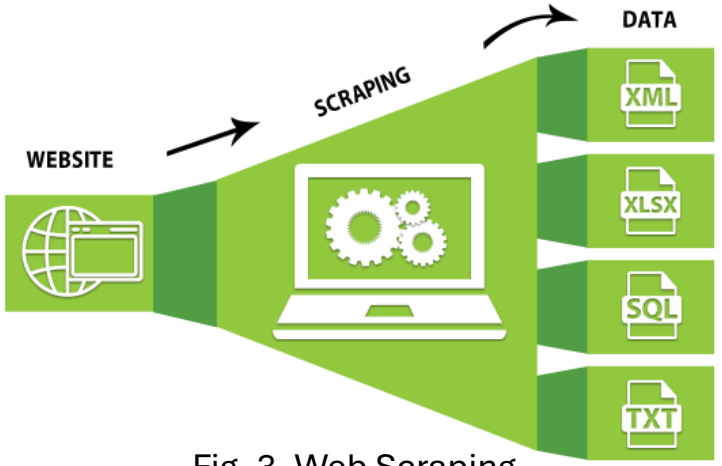


Fig. 3. Web Scraping.



Fig. 4. Public Data Sources.

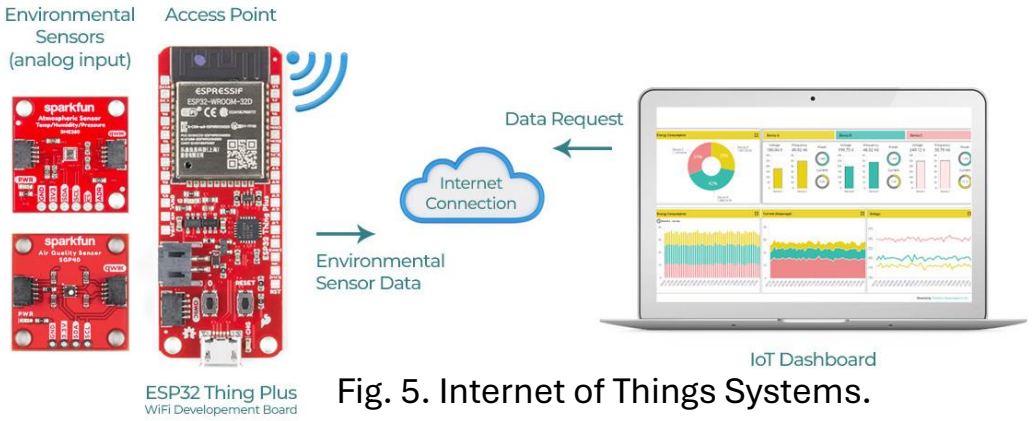


Fig. 5. Internet of Things Systems.

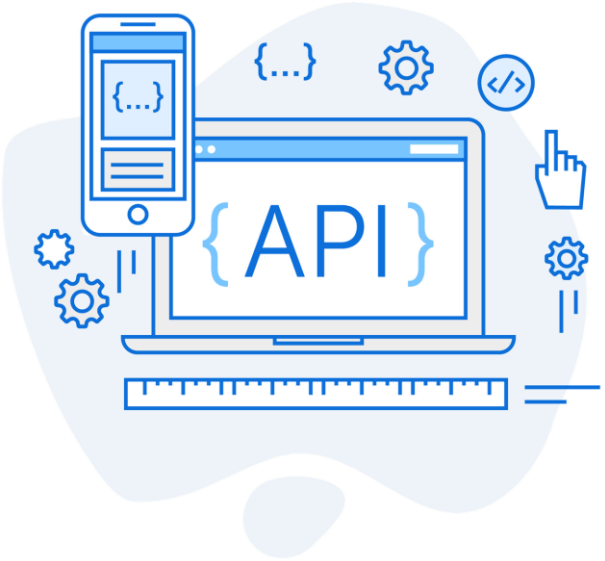
Google (2026)

Other Methods of Data Ingestion/Sourcing

- **Application Programming Interface (API):** APIs provide a structured way for applications to communicate with each other. In the context of data sourcing, many websites and services offer APIs that allow direct access to their data in a structured format, such as JSON or XML.

- This method is more efficient and reliable than web scraping because it does not require parsing HTML code, and the data structure is predefined and documented by the service provider.

- Use Cases: APIs are widely used for accessing real-time data, integrating third-party services (like social media feeds, weather data, or payment gateways), and automating data exchange between different software applications.



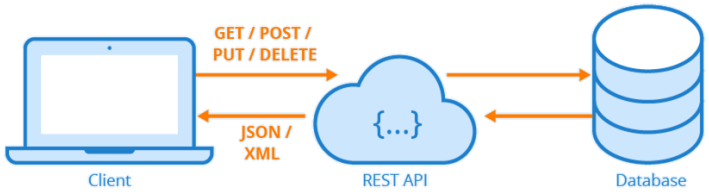


Fig. 6. API System (Google, 2026).

Ingestion Approaches: Batch vs Streaming

Batch Ingestion

- Large volumes processed at scheduled intervals.
- High throughput, lower real-time requirements.

- Examples: Daily sales reports, monthly financial analysis.

Streaming Ingestion

- Continuous data flow processed in real-time.
- Low latency, immediate insights required.

- Examples: Fraud detection, IoT monitoring, clickstreams.

Popular Sources of Datasets

Source	Example	Strength	Risk
Published file	ONS dataset as CSV	Stable, documented, citable	Updated rarely; one shape only
Official portal	data.gov.uk, data.police.uk	Clear, often open licence	Coverage varies by publisher
API	Open-Meteo, GitHub, TfL	Current, filterable, structured	Keys, limits, and versions change
Web page	A retailer's price list	Data found nowhere else	Fragile; legally uncertain
Internal database	A company's sales system	Complete and detailed	Access, confidentiality, personal data

Compare on four things: reliability, cost, coverage, and license.

Choose the least fragile route

1

Bulk file or portal

Stable, documented, clear licence

2

Official API

Current and filterable, with known limits

3

Scraping

Only if 1 and 2 do not exist and the terms and conditions allow it

4

Internal data

Only with permission and an agreement

Scraping is the last resort, not the default.

Licence, provenance, personal data

Licence

What may I do with it?

- Read the licence or terms of service
- Publicly visible is not free to reuse
- Open Government Licence, CC BY: attribute
- Many API terms forbid redistribution

Provenance

Can someone repeat it?

- Source, URL and parameters
- Collection time, in UTC
- Raw data kept unchanged
- Code and provenance record committed (data ignored).

Personal data (UK GDPR)

Is anyone identifiable?

- Names, usernames, photos, posts all count
- Pseudonymised is still personal data
- Lawful basis; collect only what you need
- Check whether ethics review is needed first

THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

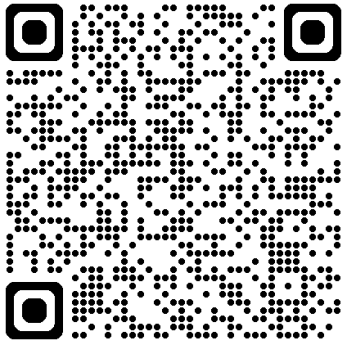
COLLECTING DATA FROM APIS

SOURCES, HTTP, JSON, KEYS, LIMITS, AND TURNING RESPONSES INTO TABLES

What is the temperature outside this building right now?

Give me one line of code.

You just used an API



`https://api.open-meteo.com/v1/forecast?latitude=51.5144&longitude=-0.1165&current=temperature_2m`

scheme | host | endpoint path | query parameters: key=value, joined by &


```
{
  "latitude": 51.51,
  "longitude": -0.12,
  "current_units": {"temperature_2m": "°C"},
  "current": {
    "time": "2026-10-05T10:00",
    "temperature_2m": 14.2
  }
}
```


A web address, some parameters, and a semi-structured answer.

Application Programming Interface (API)

APIs become essential because they provide standardised access and secure interactions with systems or applications.

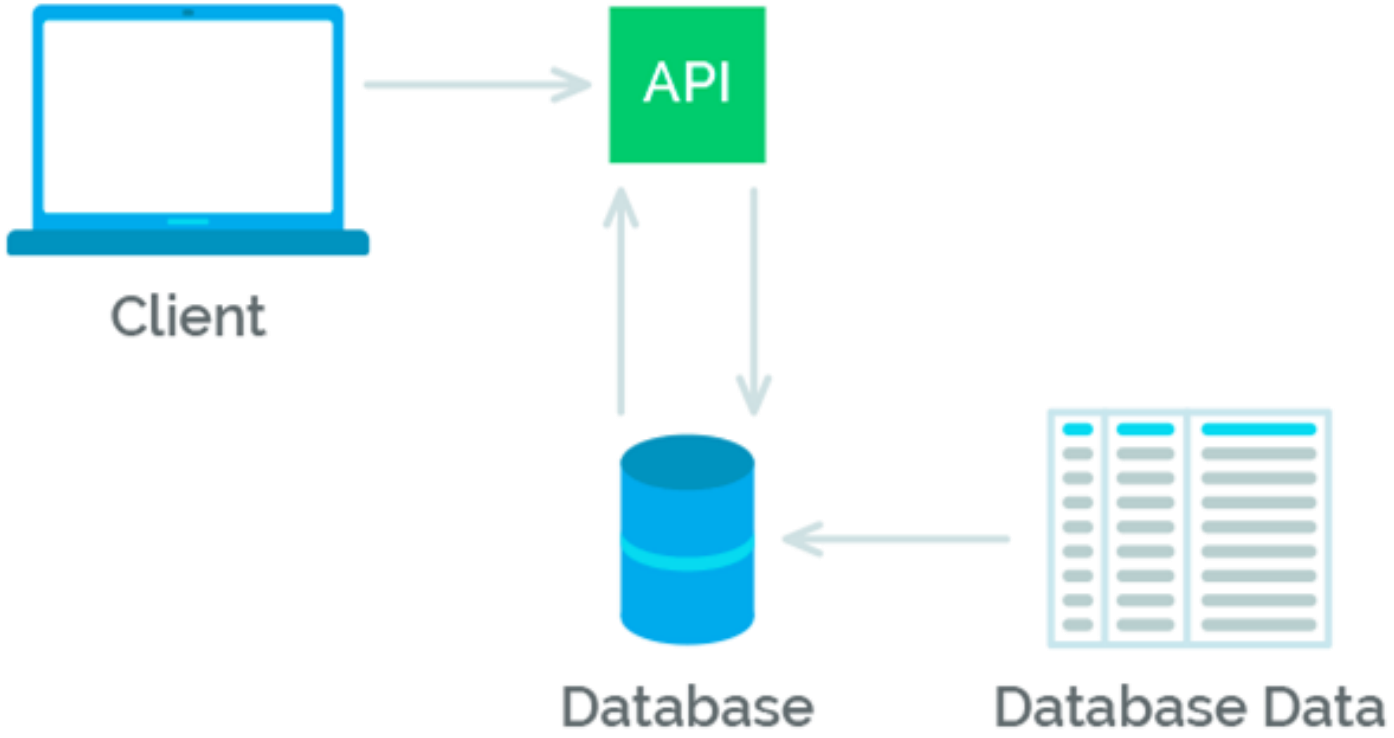


Fig. 7. API System (Google, 2026).

What is an API?



https://youtu.be/s7wmiS2mSXY?si=1UEmXDYDSNpZlRT8

What is API?

An API is the messenger between applications. It allows different software applications to communicate with each other. We can also consider APIs as "machine-to-machine middlemen". APIs can be used to query or write to a datastore, which could be internal or external.

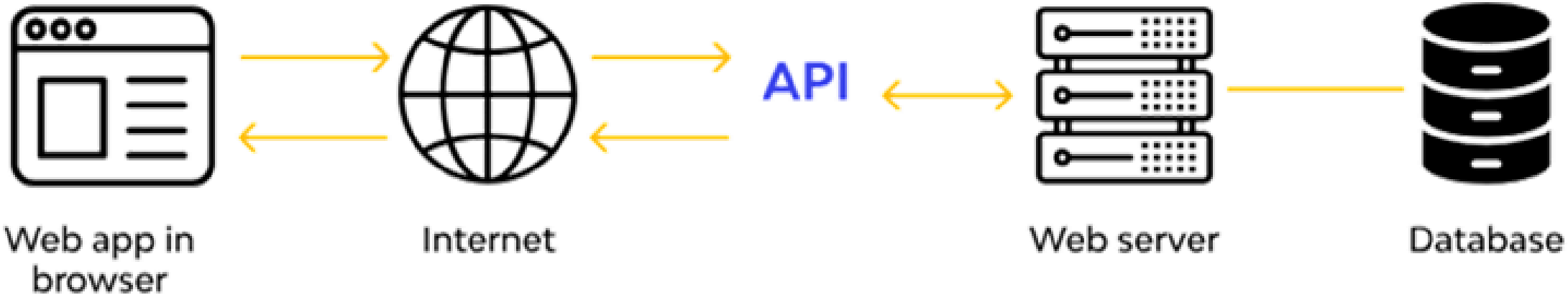


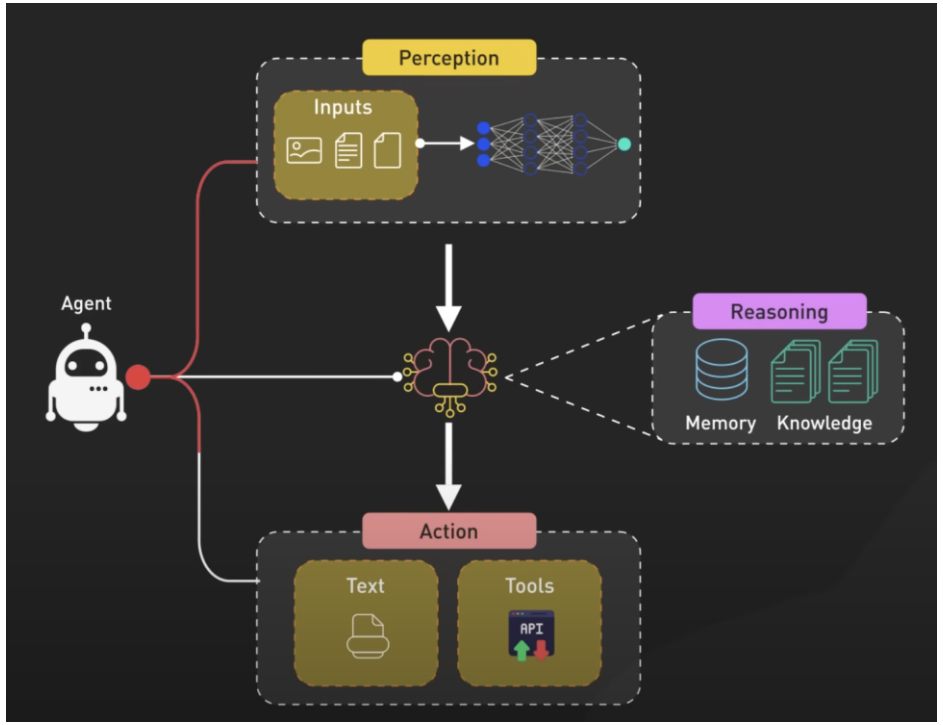
Fig. 8. API System (Google, 2026).

Why APIs Matter?

- **Scale**: APIs allow automated collection of massive datasets that would be impossible to gather manually.
- **Freshness**: Real-time or near-real-time access to constantly updating data sources.
- **Structure**: Data typically comes in consistent, structured formats (JSON, XML).
- **Efficiency**: Target exactly the data you need rather than downloading entire datasets.
- **Integration**: Combine multiple data sources into unified pipelines.

APIs in the age of AI

- AI agents, or agentic AI, are another trending topic now. APIs are one communication channel they can use.



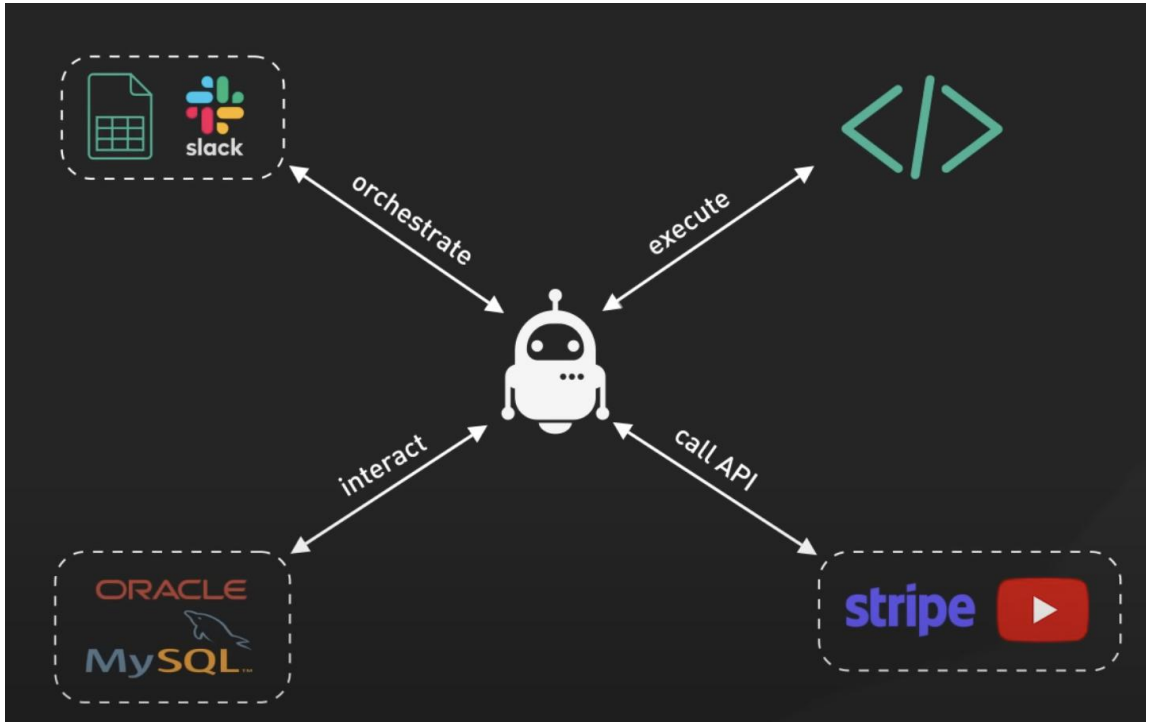


Fig. 9. Agent System (Google, 2026).

https://www.youtube.com/watch?v=eHEHE2fpnWQ

API Fundamentals

Many APIs are built using RESTful protocols. REST (Representational State Transfer) is a set of guidelines that standardises communication between applications/services.

Some initial terminology for the key elements of an API interaction:

The client: The program that is connecting to the API. This is the app/service requesting information or requesting that information be added.

The request: What the client is requesting from the API.

The resource: The information or object the client is requesting. This could be flight availability (as in the video), a photo from Facebook, or pretty much anything that may be served over the internet.

The server: The place where the resource lives (e.g. the database we want to query or update).

The response: What the server sends back over the API in response to the request made by the client (e.g. the resource).



API Economy

- According to research firm Gartner, “APIs make it easier to integrate and connect people, places, systems, data, and algorithms, create new user experiences, share data and information, authenticate people and things, enable transactions and algorithms, leverage third-party algorithms, and create new products/services and business models.”

- Kristin R. Moyer, vice president and distinguished analyst at Gartner, said: “The API economy is an enabler for turning a business or organisation into a platform. That is to say, the API economy is achieved when firms expose business assets via (web/REST) APIs to third parties to gain part of the created value through revenue sharing or other mechanisms.”

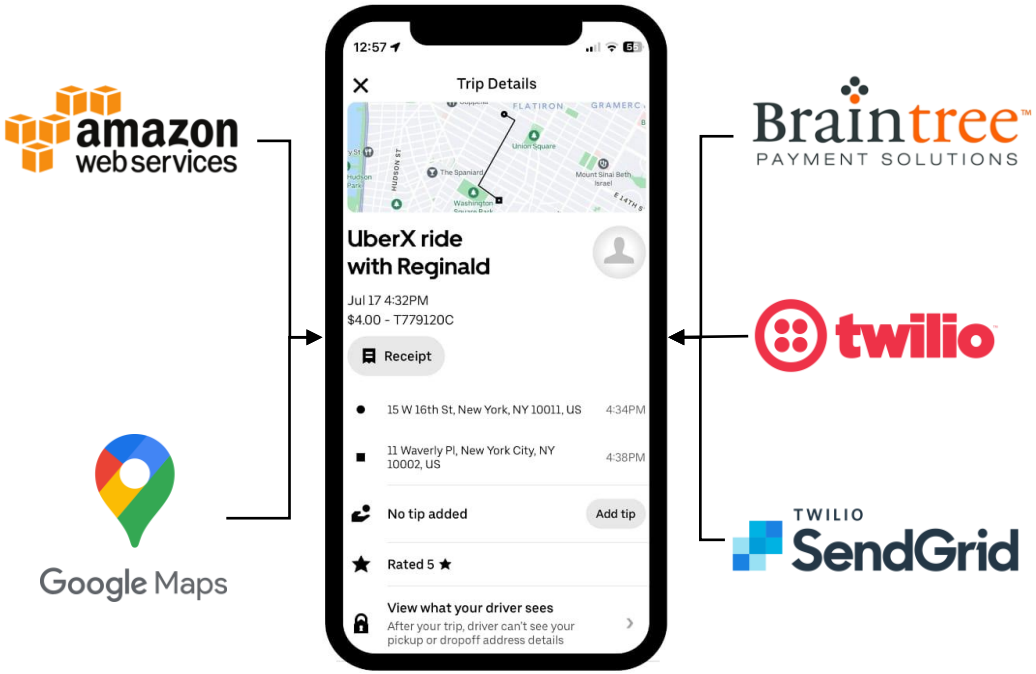


Fig. 10. API economy case – Uber (Google, 2026).

(Medium, 2016) The Secret Behind Uber’s Industry Disruption: The API-Economy | by Alexander Braun | Medium

API Fundamentals

REST APIs then communicate using HTTP as a protocol. HTTP allows four basic methods:

GET - to request a specific piece of information;
POST - to create a new resource;
PUT - to edit or update a resource;
DELETE - to delete a resource.

For example, the client may ask the server to GET the flight price of a given flight. After this, the client may request the server to POST a new customer booking record into their system (i.e. buy a flight ticket). We could also imagine future events such as updating the passenger's name (a PUT request) or cancelling the booking (a DELETE request). Note these would all be separate requests … each request will use only one HTTP method.

The actual client request may look something like this:

```
GET https://api.fakesite.com/v1?id=1234
```

→ GET request + API Endpoint

(An endpoint is a specific URL where an API can be accessed)

An API is a set of endpoints

`https://data.police.uk/api/crimes-street/all-crime?lat=51.5144&lng=-0.1165&date=2026-07`

Base URL

the API itself

Endpoint path

one kind of resource: street-level crimes

Query parameters

filter it: where and which month

Path parameters pick a thing (street-level crimes). Query parameters filter or shape the result (?date=2026-07).

LSE THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

Methods and the status codes worth recognising

Methods

`GET`	read
`POST`	create
`PUT / PATCH`	replace / change
`DELETE`	remove

Status codes: read the first digit

`2xx`	**Success**	`200 OK · 201 Created`
`3xx`	**Redirection**	`301 Moved · 304 Not modified`
`4xx`	**Your request is wrong**	`400 · 401 · 403 · 404 · 429`
`5xx`	**The server has a problem**	`500 · 502 · 503`

4xx: change your request. 5xx: wait and retry. 429: slow down.

Query parameters and headers, in Python

```
import requests

r = requests.get(
    "https://api.open-meteo.com/v1/forecast",
    params={"latitude": 51.5144, "longitude": -0.1165,
            "current": "temperature_2m"},
    headers={"User-Agent": "ST445 lecture demo (LSE)"},
    timeout=20)

r.status_code                 # 200
r.url                         # query string built for you
r.headers["Content-Type"]     # application/json
r.json()["current"]
```

`params=`
Let requests build and encode the query string

`timeout=`
Never wait forever for a reply

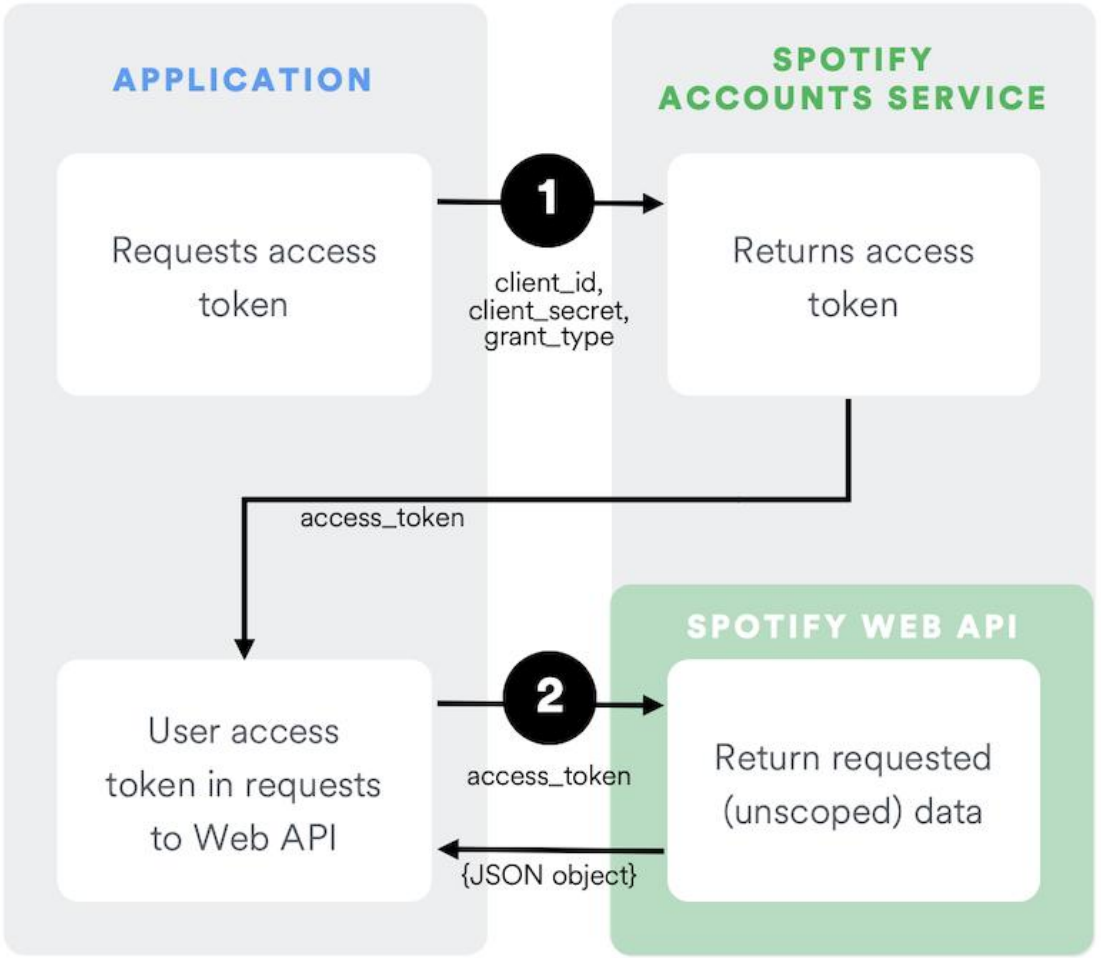
`User-Agent`
Say who you are; some servers refuse anonymous scripts

Predict
200, 404 or 401? The demo asks before it runs

API Authentication

API authentication is the process of verifying the identity of clients attempting to access an API. It's a crucial security mechanism that ensures only authorised users or applications can interact with your API endpoints.

- **API Keys**: Simple string sent with requests (i.e., https://api.example.com/data?api_key=YOUR_API_KEY)
- **Bearer Tokens**: Authentication tokens, often from OAuth. Tokens containing encoded user and permission information.
- **Basic Auth**: Username/password encoded in headers.
- **OAuth 2.0**: Standard protocol for authorisation. Provides access tokens after authentication via an auth server.



Reading documentation you have never seen

1	**Base URL and access**	Is a key needed? Any sign-up?
2	**The endpoint that fits**	Match the description to your question
3	**Parameters and formats**	Required or optional? Dates as YYYY-MM?
4	**One example response**	Read it before writing any code
5	**Limits**	Rate limits, maximum results, coverage, lags

Today: data.police.uk/docs. How many crimes occurred near the LSE last month?

Demo: crimes near the LSE

```
# 1. Discover: which months are published?
dates = requests.get(
    "https://data.police.uk/api/crimes-street-
dates").json()
latest = dates[0]["date"]          # e.g. "2026-07"

# 2. Fetch: street-level crimes near the LSE
crimes = requests.get(
    "https://data.police.uk/api/crimes-street/all-crime",
    params={"lat": 51.5144, "lng": -0.1165, "date":
latest},
    timeout=30).json()

len(crimes)
print(json.dumps(crimes[0], indent=2))
```

Discover, then fetch

Ask the API what exists before asking for it. Guessing a month that is not published returns nothing.

Licence

data.police.uk is published under the Open Government Licence: free to reuse with attribution.

API Response


```
{
  "business_id": "PK6aSizckHFWk8i0oxt5DA",
  "full_address": "400 Waterfront Dr E\nHomestead\nHomestead, PA 15120",
  "hours": {},
  "open": true,
  "categories": [
    "Burgers",
    "Fast Food",
    "Restaurants"
  ],
  "city": "Homestead",
  "review_count": 5,
  "name": "McDonald's",
  "neighborhoods": [
    "Homestead"
  ],
  "longitude": -79.910032,
  "state": "PA",
  "stars": 2,
```




JSON is semi-structured data

JSON	Example	Python	pandas
object	`{"name": "Houghton St"}`	`dict`	columns, after flattening
array	`[1, 2, 3]`	`list`	rows, or a list in a cell
string	`"2026-07"`	`str`	`object / string`
number	`51.5144`	`int / float`	`int64 / float64`
true / false	`true`	`bool`	`bool`
null	`null`	`None`	`NaN / None`

Objects nest inside objects and arrays: that is what makes JSON semi-structured.

Three JSON traps

```
first = crimes[0]
first["location"]["street"]["name"]
# 'On or near Houghton Street'

type(first["location"]["latitude"])
# <class 'str'>   a string, not a number

first["outcome_status"]
# None for some crimes, a dict for others
```

No date type

Dates arrive as strings: parse them as ISO 8601

Numbers that are labels

Large IDs lose digits as floats

Missing is not null

A key that is absent ≠ a key set to null

Where keys belong

In Colab

```
from google.colab import userdata
token = userdata.get("GITHUB_TOKEN")
headers = {"Authorization": f"Bearer {token}"}
```

On your own machine

```
# .env (listed in .gitignore)
GITHUB_TOKEN=...

import os
from dotenv import load_dotenv
load_dotenv()
token = os.environ["GITHUB_TOKEN"]
```

In a header, not the URL

URLs end up in logs, history and printed output

Outside the code

Secrets panel or environment variable

Never printed, never committed

.env is in .gitignore

Leaked? Revoke it

Make a new key; deleting the commit is not enough

Pagination: results come in pages




```
Link: <…&page=2>; rel="next", <…&page=31>; rel="last"
```

```
url = "https://api.github.com/repos/pandas-dev/pandas/issues"
params = {"per_page": 100, "state": "open"}
issues = []
for _ in range(3):
    r = requests.get(url, params=params, timeout=20)
    issues.extend(r.json())
    url = r.links.get("next", {}).get("url")
    params = None           # the next URL already has them
    if not url:
        break
    time.sleep(1)
```

Read what comes back

GitHub counts every pull request as an issue, so the issues endpoint returns both. The docs say so; the data shows it.

Rate limits and retrying with backoff

GitHub, per hour

60 without a token

5,000 with a token

Read X-RateLimit-Remaining before you loop.

```
RETRYABLE = {429, 500, 502, 503, 504}

def get_with_backoff(url, params=None, tries=5):
    for attempt in range(tries):
        r = requests.get(url, params=params,
timeout=30)
        if r.status_code not in RETRYABLE:
            r.raise_for_status()   # other 4xx: fail
now
            return r
        wait = float(r.headers.get("Retry-After",
                                   2 ** attempt))
        time.sleep(wait + random.random())
    r.raise_for_status()
```

Waits of about 1, 2, 4, 8, 16 seconds.

Caching: never fetch the same thing twice

```
def cached_get(name, url, params=None):
    path = CACHE / f"{name}.json"
    try:
        r = requests.get(url, params=params, timeout=20)
        r.raise_for_status()
        path.write_bytes(r.content)      # save as
received
        return r.json()
    except requests.RequestException:
        return json.loads(path.read_text())    # fall
back
```

Faster
Re-running a notebook does not re-download

“Kinder”
Fewer requests to someone else's server

Reproducible
The saved response is your raw data, with a timestamp

In your project, the cache is your raw data: keep it in data/raw/, unchanged, and out of Git.

From nested JSON to a table

```
import pandas as pd

crimes_df = pd.json_normalize(crimes, sep="_")
crimes_df[["category", "month",
           "location_street_name",
           "outcome_status_category"]].head()

crimes_df["location_latitude"] = pd.to_numeric(
    crimes_df["location_latitude"])   # was text
```

`location → street → name`



`location_street_name`

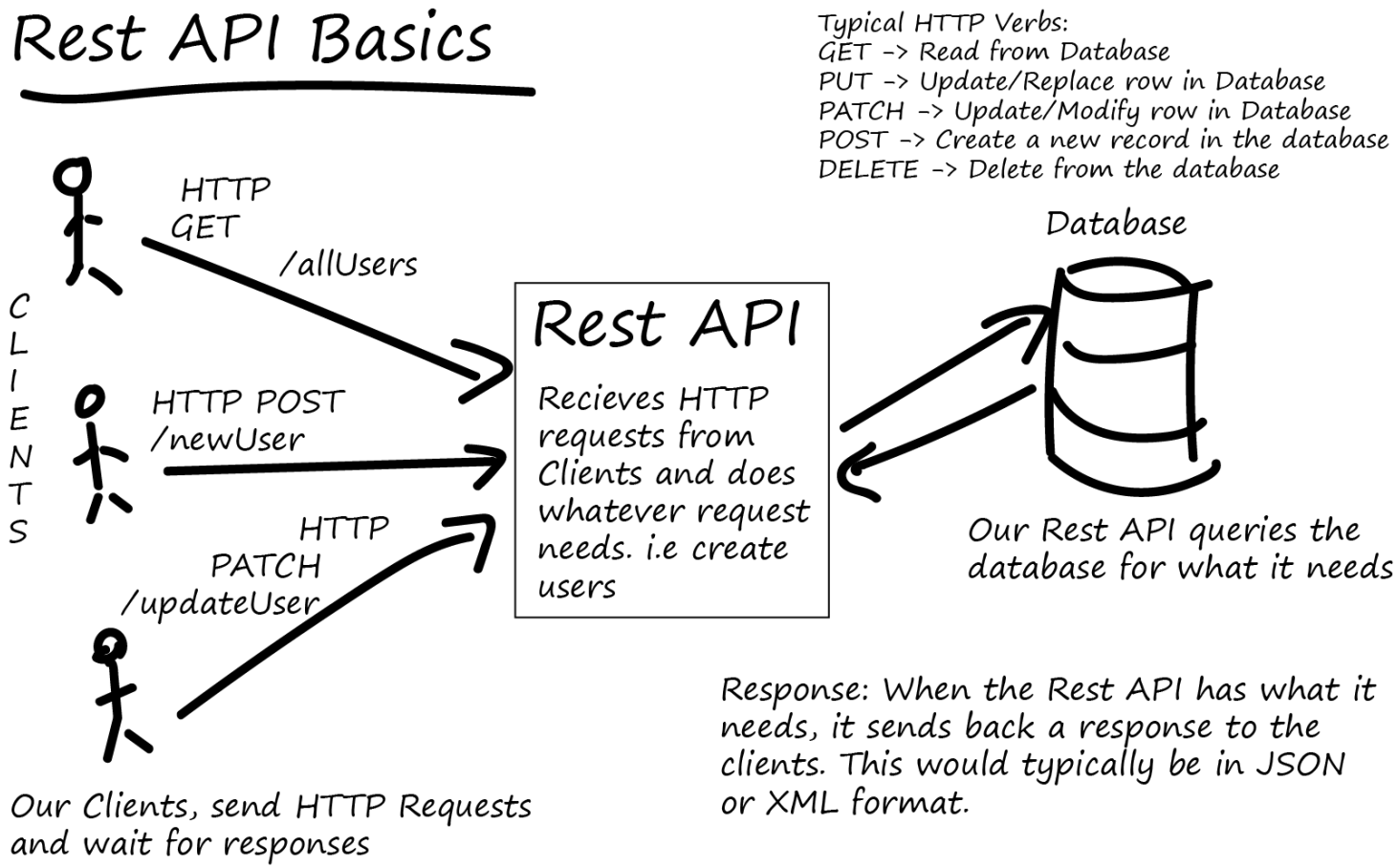
Write this first

"One row is one crime."

It is the first line of your data dictionary.

json_normalize always gives you a table.

API Fundamentals – Wrap-up



RESTful APIs are like ordering at a casual restaurant:

The menu is like API documentation; the waiter is the API. You follow the menu to order a specific dish (GET request), sometimes you provide special instructions (query parameter), and you might need to show ID to order alcohol (authentication). You can only order a specific number of dishes at once (rate limits).

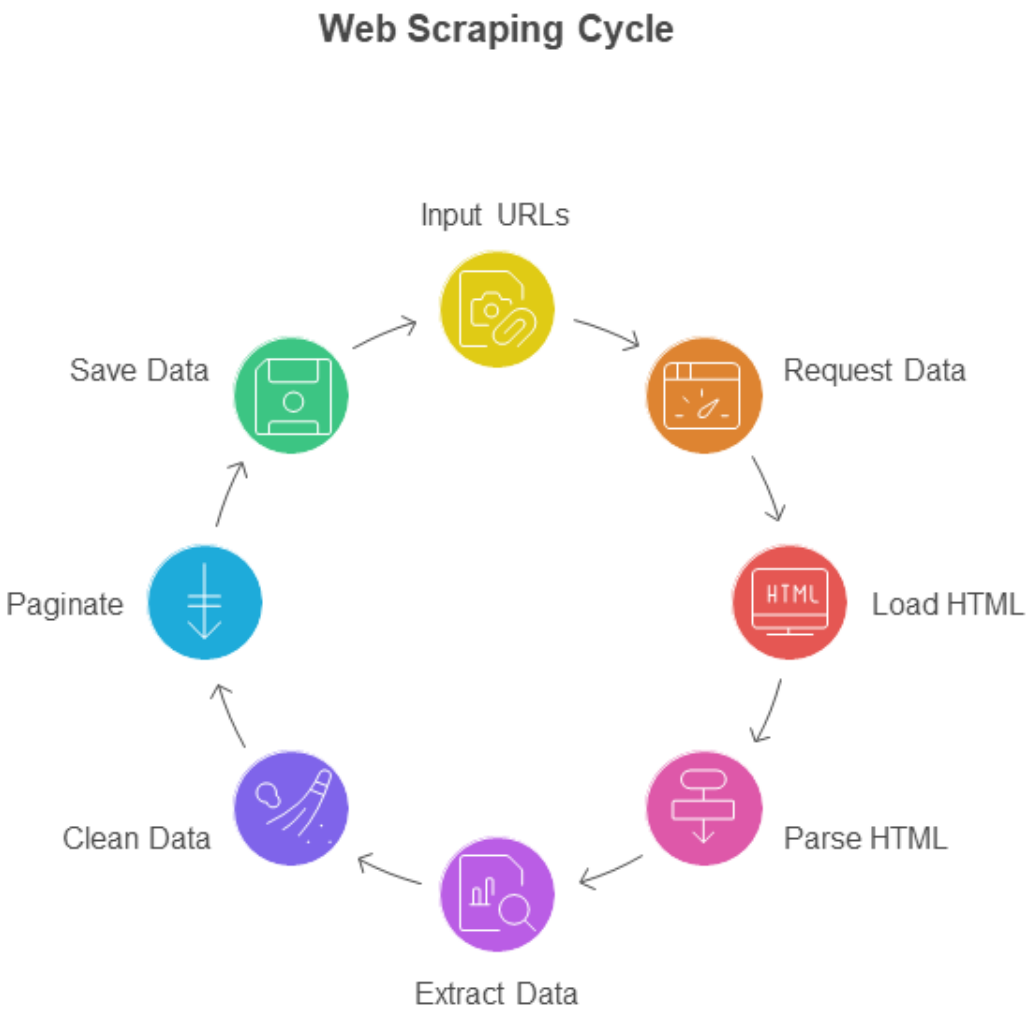
Summary

- We introduce the Data Pipeline Ingestion Stage.
- We introduce the APIs.
- We introduce the API's status code.
- We introduce the JSON semi-structured output format.

THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

WEB SCRAPING

Web Scraping



Web scraping is a process in which data, or any essential piece of information, is automatically collected from a website for the purpose of enabling humans, programs, or AI models to make more informed decisions.

(Scraper API, 2026)

HyperText Markup Language (HTML) is the language or script that many web pages are written in. It describes the content and structure of a web page so that a browser is able to render it for the viewer.

HTML Example

HTML consists of various tags written in angular brackets, mostly in opening and closing pairs:

- <html> and </html> Enclosed code is interpreted as HTML.
- <head> and <body> Browser tab and browser content areas.
- <title> Defines the text that appears within the tab or window heading area.
- <h1> , <h2> , <h3> … Heading styles.
- <p> A paragraph separated by lines.
- <a> link tag (e.g. hyperlink in the text).
- <img> Self enclosing image tag with parameters.
- and <li> define a list.
- Defines a section of a document.
- <th> Defines a table header.
- <td> Defines a cell in a table.

HTML Reference

```
<html>
    <head>
        <title>Garden Roses</title>
    </head>
    <body>
        <h1 style="font-family:Arial; color:red">Species</h1>
        <p>There are over 100 species of rose.</p>
        <!-Part b -->
        <ul>
            <li>Climbing roses</li>
            <li>Shrub roses</li>
            <li>Rambling roses</li>
        <ul>
    </body>
</html>
```

Basic web scraping steps

1. Get content from the web (Requests).
2. Extract information (BeautifulSoup).
3. Reshape and save the information as data (Pandas).

Getting to know the target website is important. You can do this by inspecting the HTML.

Use Google Chrome > More Tools > Developer Tools (or right-click and click on "Inspect") to inspect the HTML code for a website.

Python modules and the internet

Python provides some great solutions for interacting with the internet:

- ***urllib*** is the basic library module for getting internet resources
- As the Python documentation itself says, the ***Requests*** module is recommended for a higher-level HTTP client interface
  - ***Requests*** is installed as part of Anaconda Python.
  - It's one of the most downloaded external libraries for Python!
- ***BeautifulSoup*** can parse HTML to extract data.
  - Content extraction with matching using regular expressions

These two pages look identical. Why can Python read one and not the other?

quotes.toscrape.com

Quotes to Scrape

“The world as we have created it…”

“It is our choices, Harry…”

“There are only two ways to live…”

quotes.toscrape.com/js/

Quotes to Scrape

“The world as we have created it…”

“It is our choices, Harry…”

“There are only two ways to live…”

Ask Python

```
for url in ["https://quotes.toscrape.com/",
            "https://quotes.toscrape.com/js/"]:
    soup = BeautifulSoup(fetch_html(url), "html.parser")
    print(len(soup.select("div.quote")), "quotes on", url)

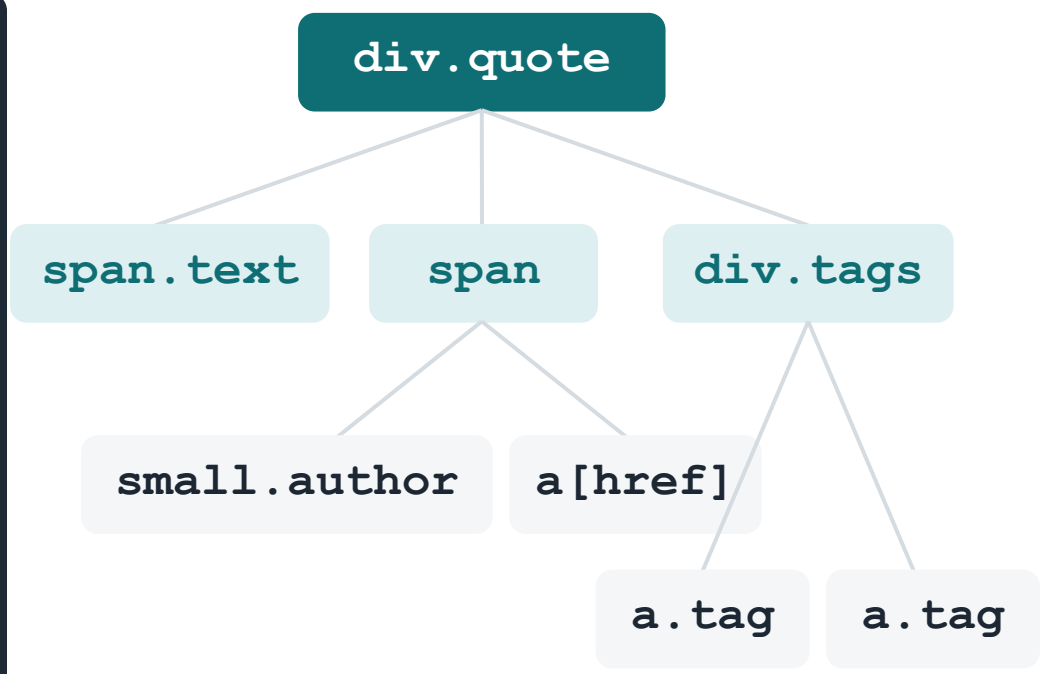
# 10 quotes on https://quotes.toscrape.com/
#  0 quotes on https://quotes.toscrape.com/js/
```

Hold that thought

Same appearance in the browser, different HTML underneath. We'll answer this after seeing how scraping works.

HTML is semi-structured markup: a tree

```
<div class="quote">
  <span class="text">“It is our choices…”</span>
  <span>by <small class="author">J.K.
Rowling</small>
    <a href="/author/J-K-Rowling">(about)</a>
  </span>
  <div class="tags">
    <a class="tag"
href="/tag/abilities/">abilities</a>
    <a class="tag"
href="/tag/choices/">choices</a>
  </div>
</div>
```



Tag · attributes (class, href) · text

Same idea as nested JSON, different syntax: you navigate it by tags and attributes.

Get to know the page before you write code

1	**Right-click → Inspect**	Opens the browser's developer tools on that element
2	**Hover and expand**	Watch the page highlight as you move through the tree
3	**Note tags and classes**	div.quote, span.text, small.author: these become your selectors
4	**View source as well**	Source = what the server sent. Inspect = the page after scripts ran.

The difference between Source and Inspect is the answer to the opening question.

LSE THE LONDON SCHOOL OF ECONOMICS AND POLITICAL SCIENCE

CSS selectors and XPath

To select	CSS selector	XPath
Every with class quote	`div.quote`	`//div[@class="quote"]`
The text span inside each quote	`div.quote span.text`	`//div[@class="quote"]/span[@class="text"]`
Element with id "main"	`#main`	`//*[@id="main"]`
Links that have an href	`a[href]`	`//a[@href]`
Tag links inside div.tags	`div.tags a.tag`	`//div[@class="tags"]/a[@class="tag"]`

```
quote.select_one("span.text").get_text()      # text
quote.select_one("a[href]")["href"]           # attribute
```

A scraping recipe

1 — Check

API or download? robots.txt and terms?

2 — Fetch

requests, with a User-Agent and pauses

3 — Parse

BeautifulSoup builds the tree

4 — Extract

Selectors, safely, into dictionaries

5 — Save

Raw HTML, a table, a provenance record

Requests → BeautifulSoup → pandas.

Extracting text: titles and prices

```
BOOKS = "https://books.toscrape.com/catalogue/page-1.html"
soup = BeautifulSoup(fetch_html(BOOKS), "html.parser")
pods = soup.select("article.product_pod")
len(pods)                                    # 20

for pod in pods[:3]:
    print(pod.select_one("h3 a").get_text(),
          pod.select_one("p.price_color").get_text())

# A Light in the ...    £51.77
# Tipping the Velvet    £53.74
# Soumission            £50.10
```

select / select_one

All matches, or the first match, for a CSS selector

get_text(strip=True)

The visible text, with surrounding whitespace removed

Truncated titles

The page shortens them. Look for the full value somewhere else

Extracting attributes

```
<article class="product_pod">
  <p class="star-rating Three"></p>
  <h3><a href="a-light-in-the-attic_1000/index.html"
         title="A Light in the Attic">A Light in the ...</a></h3>
  <p class="price_color">£51.77</p>
</article>
```

```
link = pod.select_one("h3 a")
link["title"]     # 'A Light in the Attic'
link["href"]      # a relative URL
pod.select_one("p.star-rating")["class"]
# ['star-rating', 'Three']
```

The rating is a class name

CSS draws the stars. The data is the word Three, which you map to the number 3.

Missing or inconsistent elements

```
def text_or_none(node, selector):
    found = node.select_one(selector)
    return found.get_text(strip=True) if found else None

def parse_price(text):
    if text is None:
        return None
    digits = re.sub(r"[^0-9.]", "", text)    # drops £ (and Â)
    return float(digits) if digits else None

parse_book(broken_pod)
# {'title': None, 'price_gbp': None, 'rating': None, ...}
```

A missing node is None

None.get_text() crashes the run; a helper returns None instead

Convert in one place

Text to numbers and categories, once, with tests

£ becomes Â£

No charset sent: requests guesses ISO-8859-1. Set r.encoding = "utf-8"

Following pages, politely

```
rows, url = [], BOOKS
for _ in range(3):                          # only what we
need
    soup = BeautifulSoup(fetch_html(url), "html.parser")
    rows.extend(parse_book(p)
                for p in
soup.select("article.product_pod"))
    nxt = soup.select_one("li.next a")
    if not nxt:
        break
    url = urljoin(url, nxt["href"])     # relative →
absolute
    # fetch_html pauses 1 second after each request

books = pd.DataFrame(rows)                  # 60 rows × 5
columns
```

Follow next

Use the page's own link, not invented page numbers

urljoin

page-2.html becomes a full URL

Pause

One request a second is plenty for a project

Stop early

Collect what your question needs

When the data is already a table

```
from io import StringIO

product = fetch_html(books.loc[0, "url"])
tables = pd.read_html(StringIO(product))
len(tables)              # 1
tables[0]
#                    0                    1
# 0                UPC  a897fe39b1053632
# 1       Product Type                Books
# 2   Price (excl. tax)              £51.77
```

Every <table> becomes a DataFrame

Last year's countries-by-population example used this. You still clean the columns afterwards: numbers with commas and % signs arrive as text.

When scraping is the wrong tool

An API or bulk download exists

Wikipedia has an API and full database dumps. Last year's character list would come more reliably from the API.

The page is built by JavaScript

requests receives the page before scripts run. The answer to the opening question: next slide.

The terms forbid it

Then do not scrape, however easy it is technically. Look for an official route.

Check before you fetch: it is step 1 of the recipe.

JavaScript-rendered pages: the answer

```
<div class="container"></div>
<script>
  var data = [{"text": "“The world as we have created it…”",
               "author": {"name": "Albert Einstein"}, ...}];
  for (var i in data) { /* build each quote in the browser */ }
</script>
```

1. Network tab

Find the JSON request the page makes, and call that instead

2. Data in the page

Sometimes it is in a script, as here; parse it with json

3. Browser automation

Playwright or Selenium run the scripts. Slower and heavier; awareness only

Try them in this order. The first is often an API in disguise.

Legal and ethical practice: five things to check

robots.txt	The site's published wishes about automated access
Terms of service	A contract; many forbid automated collection
Copyright and database rights	Selection and arrangement can be protected, as can whole databases
Personal data (UK GDPR)	Publicly visible is not free to use
Courtesy	Identify yourself, pause, collect only what you need, cache

robots.txt: read it first

```
# https://example.org/robots.txt
User-agent: *
Disallow: /search
Disallow: /account/
Crawl-delay: 10

User-agent: GPTBot
Disallow: /
```

```
from urllib.robotparser import
RobotFileParser

rp = RobotFileParser(
    "https://www.gov.uk/robots.txt")
rp.read()
rp.can_fetch(HEADERS["User-Agent"],

"https://www.gov.uk/search/all")
```

Not a law, but a clear signal

Ignoring it is poor practice and weakens any defence

Missing is not permission

No robots.txt means no stated wishes; the terms still apply

Copyright, database right and personal data

Copyright and database right

- Facts are not protected; their selection and arrangement, and text such as reviews, can be
- The UK has a separate database right
- Section 29A CDPA 1988: text and data mining for non-commercial research, on material you can lawfully access

Personal data (UK GDPR)

- Names, usernames, photos and posts are personal data, even when public
- You need a lawful basis, and collect only what you need
- Usually needs research ethics review before collecting

Access

- Never bypass a login, paywall or other access control
- Never scrape with someone else's account
- If you need to log in to see it, find an official route

Scraping in the age of generative AI

Sites are closing the door

robots.txt files now name AI crawlers such as GPTBot, ClaudeBot and CCBot; some platforms charge for API access

Bartz v. Anthropic (US)

June 2025: training on lawfully bought books was fair use; downloading and keeping pirated copies was not. Settled for $1.5bn; final approval July 2026

Not settled law

One US district court ruling; dozens of AI copyright cases continue

The lesson for your project

The same use was lawful or not depending on how the data was acquired.

Record how you acquired every dataset. That record is your provenance.

Courtesy, in code

ILO3

```
HEADERS = {"User-Agent":
           "ST445 project (you@lse.ac.uk)"}

for url in urls_you_need:
    if not allowed(url):        # robots.txt
        continue
    html = fetch_html(url)      # cached, pauses
    ...
```

Say who you are

A User-Agent with a contact address

Slow down

A pause between requests; off-peak for big runs

Take only what you need

Fewer pages, fewer fields, less personal data

Never twice

Cache every page you fetch

Storing what you collect

```
st445-project/
├── data/
│   ├── raw/           exactly as received, never edited
│   │   └── books/page_1_2026-10-05T101500Z.html
│   ├── processed/     written by code, from raw/
│   └── provenance.jsonl   one line per collection run
├── src/collect.py     the collection code
└── .gitignore         data/raw/*  data/processed/*
```

```
{"collected_at": "2026-10-05T101500Z",
 "source": "books.toscrape.com",
 "pages": 3, "files": [...sha256...],
 "user_agent": "ST445 ...",
 "robots_txt_checked": true,
 "pause_seconds": 1.0,
 "terms": "Sandbox for scraping practice"}
```

Immutable raw data

Never edit a raw file; clean in code

When

Collection time in UTC, ISO 8601

What

URL, parameters, pages; never a key

Under what terms

Licence, terms, robots.txt checked

In Git

Code and provenance record: yes. Raw data: no

Summary

- We introduce the concept of web scraping.
- We introduce the HTML structure.
- We introduce the API's status code.

References

ScraperAPI (2026) *The Web scraping coach: Your no-nonsense guide to scraping the web*. Available at: https://www.scraperapi.com/web-scraping/ (Accessed: 02 October 2026).